

DOCKER MASTERY

FROM ZERO TO PRODUCTION EXPERT

A complete hands-on guide for ML Engineers moving into DevOps/MLOps

7

chapters

50+

code examples

6

real projects

1

complete book

Table of Contents

Chapter 1 — The Real Foundation — How Docker Works Inside

- › Linux namespaces & cgroups
- › Union filesystem & layers
- › The Docker daemon
- › Hands-on: see it yourself

Chapter 2 — Dockerfile Mastery — Every Instruction Explained

- › ENTRYPOINT vs CMD
- › ENV vs ARG
- › WORKDIR, USER, EXPOSE
- › .dockerignore
- › Project: Dockerise your ML app

Chapter 3 — Volumes & Data — Nothing Gets Lost

- › Why containers lose data
- › Named volumes
- › Bind mounts
- › tmpfs
- › Project: Postgres persistence

Chapter 4 — Docker Networking — Simply Explained

- › Custom bridge & DNS
- › Port publishing deep dive
- › Container-to-container communication
- › Project: 3-tier app

Chapter 5 — Docker Compose — Real Apps in One File

- › compose.yml structure
- › depends_on & health checks
- › env files & secrets
- › Project: Full ML stack

Chapter 6 — Production Docker — What Teams Actually Do

- › Health checks
- › Resource limits
- › Restart policies
- › Registry & image tagging
- › Security basics

Chapter 7 — Debugging & Troubleshooting

- › Reading logs
- › Exec into containers
- › Inspect everything
- › Common failure patterns & fixes

The Real Foundation

How Docker actually works inside Linux

Before writing a single Dockerfile, you need to understand what Docker actually *is* at a fundamental level. Most people skip this and struggle later when things break unexpectedly. This chapter gives you the mental model that makes everything else click.

The Restaurant Analogy

Imagine you're running a restaurant. Normally, every chef works in the same shared kitchen. If one chef burns something, the smoke fills the entire kitchen. If one chef uses all the olive oil, everyone else has none. Containers solve this by giving each chef their own **private kitchen booth** — same building, same gas supply, but completely isolated space. That's exactly what Linux namespaces do for processes.

1.1 What Is a Container — Really?

A container is **not** a virtual machine. It's not a separate operating system. A container is just a **regular Linux process** running on your computer — but with two kernel features wrapping it:

- **Namespaces** — create the illusion of isolation (your own filesystem, network, process list)
- **cgroups** (control groups) — limit how much CPU, RAM, and disk a process can use

That's the entire secret. No hypervisor, no hardware emulation, no separate kernel. Docker is essentially a tool that makes it easy to set up [namespaces and cgroups](#), then start a process inside them.

Linux Namespaces — The 6 Walls of Isolation

[Linux has 6 types of namespaces](#). Docker uses all of them together:

Namespace	What it isolates — in plain English
pid	The container sees only its own processes. It thinks it's the only thing running. PID 1 inside the container is just a normal process from the host's view.
net	The container gets its own network stack — its own eth0 , its own IP address, its own iptables rules.
mnt	The container gets its own filesystem view. It sees / as its root, completely unaware of the host's real /
uts	The container can have its own hostname, different from the host machine.
ipc	Isolated inter-process communication — shared memory, semaphores.
user	Map container's root (uid 0) to a non-root user on the host — for extra security.

cgroups — The Resource Governor

While namespaces create isolation, cgroups enforce **resource limits**. Without cgroups, one container could eat all your RAM and crash your machine. With cgroups, you say: 'this container gets at most 512 MB RAM and 0.5 CPU cores.'

What cgroups look like under the hood

```
# What happens when you run:
docker run --memory=512m --cpus=0.5 myapp

# Docker creates a cgroup entry at:
# /sys/fs/cgroup/memory/docker/<container-id>/memory.limit_in_bytes
# /sys/fs/cgroup/cpu/docker/<container-id>/cpu.cfs_quota_us

# The kernel enforces these limits at the hardware level
# If the app tries to use more RAM, it gets killed (OOMKilled)
```

1.2 The Union Filesystem — Why Layers Exist

The Transparent Slides Analogy

Think of a Docker image like a stack of transparent slides on an overhead projector. Each slide adds something. The bottom slide is the base OS. The next adds Python. The next adds your packages. The last adds your code. The projector shows all slides combined — that's what the container sees as its filesystem. Crucially, if two images share the same bottom slides, Docker only stores those slides **once**.

Docker uses a storage driver called **overlay2 (on modern Linux)**. It has two layers:

- **Lower dir (read-only)** — all the image layers stacked, immutable
- **Upper dir (read-write)** — a thin writable layer unique to each running container
- **Merged view** — what the container sees: lower + upper combined

When a container modifies a file from a lower layer, overlay2 copies that file up to the upper dir first — then modifies it there. This is called **copy-on-write (CoW)**. The original in the lower dir is untouched, which is why all containers from the same image start with an identical filesystem.

Inspect overlay2 mounts directly

```
# See the overlay2 mounts on your system
docker run -d --name myapp nginx

# Find the container's overlay mount
docker inspect myapp | grep -A10 GraphDriver

# You'll see something like:
# LowerDir: /var/lib/docker/overlay2/abc.../diff
```

1.3 The Docker Architecture — 3 Pieces

Docker is not one program. It's three components talking to each other:

Component	What it does
docker CLI	The command you type. It's just a client that sends API requests. It has no power on its own.
Docker daemon (dockerd)	The background service that does all the real work — creating namespaces, pulling images, managing networks.
containerd + runc	The actual container runtime. dockerd delegates to containerd, which uses runc to call the Linux kernel and create the namespaces/cgroups.

When you type **docker run nginx**, this happens:

1. Your CLI sends a POST request to the Docker daemon via a [Unix socket at /var/run/docker.sock](#)
2. dockerd pulls the nginx image if not cached, then tells containerd to create a container
3. containerd calls runc, which calls the Linux kernel to create 6 namespaces and a cgroup
4. runc starts the nginx process inside those namespaces
5. Your terminal is connected back to that process's stdin/stdout

1.4 Hands-On: See It Yourself

Don't just read this — run these commands. Watching Docker in action at the kernel level makes everything real.

Inside the container

```
# Terminal 1: Start a container and keep it running
docker run -it --name demo ubuntu bash

# Inside the container, check your process list
ps aux
# You only see bash and ps – the container's isolated process view

# Check your network
ip addr
# You see eth0 with a 172.17.x.x address – NOT the host's network

# Check your hostname
hostname
# Shows the container ID, not your machine name
```

From the host — the container is just a process

```
# Terminal 2 (on your HOST machine, not inside container):

# Find the container's PID on the host
docker inspect demo --format '{{.State.Pid}}'
# e.g., returns 12345
```

Do This Exercise Now

This exercise is the single most important thing in this chapter. Once you see the container process in the host's process table, Docker stops feeling like magic and becomes a system you truly understand.

1.5 Containers vs Virtual Machines — The Real Difference

Aspect	Virtual Machine	Container
What it runs	A full separate OS kernel	A process on the host kernel
Boot time	30–120 seconds	Milliseconds
Size	Gigabytes (full OS)	Megabytes (just the app + libs)
Isolation	Hardware-level (hypervisor)	Kernel-level (namespaces)
Overhead	High — dedicated CPU/RAM	Near zero — shared kernel
Security boundary	Strong — separate kernel	Weaker — shared kernel
Use case	Different OS, strong isolation	Same OS, fast and lightweight

Security Consideration

Containers share the host kernel. This means a kernel exploit could potentially escape container isolation. For workloads requiring strong security boundaries (financial transactions, multi-tenant user code execution), consider VMs or tools like gVisor/Kata Containers that add a layer between the container and the kernel.

Dockerfile Mastery

Every instruction explained with intuition and examples

You've seen Dockerfiles before. Now we go deeper — not just what each instruction does, but *why* it exists and the subtle traps that catch even experienced engineers.

2.1 ENTRYPOINT vs CMD — The Most Confused Pair

The Job Title Analogy

Think of **ENTRYPOINT** as the **job title** and **CMD** as the **default task**. If you hire someone as a 'Python runner' (ENTRYPOINT), they always run Python. But their default script might be 'main.py' (CMD). You can tell them to run a different script without changing their job title.

The confusion comes from the fact that both can start your app. Here's the precise difference:

Instruction	What it controls	Can be overridden by?
ENTRYPOINT	The executable that always runs — the container's 'identity'	docker run --entrypoint flag (rare)
CMD	Default arguments passed to ENTRYPOINT, or the default command if no ENTRYPOINT	Anything you write after the image name in docker run

CMD alone — flexible

```
# Example 1: CMD alone (most common for general-purpose images)
FROM python:3.11-slim
CMD ["python", "main.py"]

# docker run myimage          → runs python main.py
# docker run myimage bash     → runs bash (CMD overridden)
# docker run myimage python -c 'print(1)' → runs that instead
```

ENTRYPOINT alone — the container IS python

```
# Example 2: ENTRYPOINT alone
FROM python:3.11-slim
ENTRYPOINT ["python"]

# docker run myimage          → runs python (no script = error)
# docker run myimage main.py  → runs python main.py
# docker run myimage -c 'print(1)' → runs python -c 'print(1)'
```

ENTRYPOINT + CMD — best pattern for production services

Always Use Exec Form

Always use the **JSON array (exec form)**: `["python", "main.py"]`. Never the shell form: `python main.py`. The shell form wraps your command in `/bin/sh -c`, which means your process is not PID 1 — it won't receive OS signals like `SIGTERM`, breaking graceful shutdown.

2.2 ENV vs ARG — Build Time vs Runtime

These two look similar but live in completely different moments:

ARG — build-time only

```
# ARG — only exists during the docker build process
# Use for: version numbers, build flags, private URLs
ARG PYTHON_VERSION=3.11
ARG PRIVATE_PYPI_URL
FROM python:${PYTHON_VERSION}-slim

# After the build completes, ARG values are GONE
# They do NOT appear in docker inspect, docker history (mostly),
# or inside the running container

# Pass ARG values at build time:
# docker build --build-arg PYTHON_VERSION=3.12 .
```

ENV — survives into the running container

```
# ENV — persists into the running container
# Use for: app configuration, paths, feature flags
ENV APP_PORT=8080
ENV MODEL_PATH=/models
ENV LOG_LEVEL=INFO

# Available inside the container as environment variables:
# import os; port = os.environ['APP_PORT']

# Also visible in:
# docker inspect mycontainer | grep -A20 Env

# Can be overridden at runtime:
# docker run -e LOG_LEVEL=DEBUG myimage
```

ENV Is Not Secret Storage

Never put secrets in ENV inside a Dockerfile. ENV values are baked into every image layer and visible to anyone who can run **docker inspect** or **docker history**. Use runtime injection (**docker run -e**) or BuildKit secrets instead.

2.3 WORKDIR — Never Use cd

WORKDIR vs cd

```
# WRONG — don't use RUN cd
RUN cd /app && pip install -r requirements.txt
# Each RUN starts a fresh shell. The cd in one RUN
# has NO effect on the next RUN instruction.

# RIGHT — use WORKDIR
WORKDIR /app
# Now every subsequent RUN, COPY, CMD runs from /app
# WORKDIR creates the directory if it doesn't exist
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .

# WORKDIR also affects interactive sessions:
# docker exec -it mycontainer bash
# You'll land in /app automatically
```

2.4 USER — Never Run as Root

By default, containers run as root (uid 0). This is dangerous — if your app is compromised, the attacker has root access inside the container and potentially on the host. Always switch to a non-root user for your application process.

Secure Dockerfile with non-root user

```
FROM python:3.11-slim

# Create a non-root user and group
RUN groupadd --gid 1001 appgroup && \
    useradd --uid 1001 --gid appgroup --shell /bin/bash \
        --create-home appuser

WORKDIR /app

# Install deps as root (need permissions)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy app files and set ownership
COPY --chown=appuser:appgroup . .

# Switch to non-root user for the application
USER appuser
```

2.5 EXPOSE — Documentation, Not Magic

EXPOSE does **not** publish a port. It's documentation — it tells other developers (and Docker Compose) which port your app listens on. The actual port opening happens with **-p** in `docker run`.

What EXPOSE actually does

```
EXPOSE 8080
# This means: 'my app listens on 8080'
# It does NOT mean the port is accessible from outside

# To actually publish it:
# docker run -p 8080:8080 myimage

# EXPOSE is useful because:
# 1. docker run -P (capital P) publishes all EXPOSEd ports randomly
# 2. Docker Compose reads EXPOSE to configure networking
# 3. Documentation for anyone reading your Dockerfile
```

2.6 .dockerignore — Control What Docker Sees

When you run **docker build .**, Docker sends your entire current directory to the Docker daemon as the 'build context'. If your directory has 5 GB of model weights, datasets, and logs — all of it gets sent before the build even starts. `.dockerignore` prevents this.

.dockerignore for ML projects

```
# .dockerignore — same syntax as .gitignore

# Python artifacts
__pycache__/
*.pyc
*.pyo
.pytest_cache/

# Virtual environments — NEVER send these
.venv/
venv/
env/

# ML artifacts — large files not needed at build time
models/
data/
*.pkl
*.h5
*.pt
*.onnx

# Git history
```

Check Your Build Context Size

Run `docker build --no-cache . 2>&1 | head -5` and look at the 'Sending build context' line. If it says '2.3 GB', you have a missing `.dockerignore` problem. The build context should typically be under 50 MB.

2.7 Project: Dockerise Your ML App from Scratch

Project 1 — Your First Production Dockerfile

Goal: Write a production-quality Dockerfile for a Python ML inference service.

Requirements:

- Use `python:3.11-slim` as base
- Install system deps + pip deps in one clean layer
- Run as non-root user
- Use `WORKDIR`, `EXPOSE`, `ENTRYPOINT` + `CMD` pattern
- Include a `.dockerignore`
- Keep final image under 500 MB

Verify success:

- `docker build -t myml:v1 .` builds without errors
- `docker run myml:v1` starts without root warnings
- `docker history myml:v1` shows clean layers
- `docker images myml:v1` shows reasonable size

Complete Production ML Dockerfile

```
# Complete solution – production ML inference Dockerfile
FROM python:3.11.9-slim

# Build-time variables
ARG APP_VERSION=1.0.0

# Runtime environment
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    APP_PORT=8080 \
    MODEL_PATH=/app/models

# System deps + cleanup in ONE layer
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        libgomp1 curl && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

# Create non-root user
```

Volumes & Data Persistence

Nothing gets lost — if you do it right

The Whiteboard vs Notebook Analogy

A container's internal filesystem is like a whiteboard. Every time you erase and redraw (stop and recreate the container), you start with a blank board. Volumes are like a **notebook** that sits next to the whiteboard — you write important things in the notebook, and it survives no matter how many times the whiteboard gets erased.

3.1 Why Containers Lose Data

This catches almost every Docker beginner. The container's writable layer — the 'upper dir' from our overlay2 discussion — is **tied to the container's lifetime**. When you delete a container, that layer is deleted with it. The image (read-only layers) survives, but all changes your app made at runtime are gone.

Demonstrate data loss — do this yourself

```
# PROVE IT — watch data disappear

# Step 1: Start a container and create a file
docker run -it --name test ubuntu bash
echo 'important data' > /myfile.txt
exit

# Step 2: The container is stopped but still exists — file is there
docker start test
docker exec test cat /myfile.txt
# Output: important data ← still there

# Step 3: DELETE the container
docker rm test

# Step 4: Create a new container from the same image
docker run -it --name test2 ubuntu bash
cat /myfile.txt
# cat: /myfile.txt: No such file or directory
# THE FILE IS GONE. The image never had it.
```

3.2 Named Volumes — Docker Manages the Storage

A named volume is storage that Docker manages on your host machine. It lives at **/var/lib/docker/volumes** and persists independently of any container. You refer to it by name, and Docker handles the rest.

Named volume — data survives container deletion

```
# Create a named volume
Docker Mastery — From Zero to Production Expert
docker volume create pgdata
```

```
# Use it — mount it at /var/lib/postgresql/data inside the container
```

3.3 Bind Mounts — Your Files, Inside the Container

A bind mount maps a specific directory from your host machine directly into the container. Changes on either side are **immediately visible** on both sides. This is the foundation of development workflows — edit code in your IDE, the running container sees the change instantly.

Bind mount for development workflow

```
# Mount your current directory into the container
docker run -it \
  -v $(pwd):/app \
  -w /app \
  python:3.11-slim \
  python main.py

# Syntax: -v HOST_PATH:CONTAINER_PATH

# Real dev workflow: auto-reload server
docker run -d \
  --name dev \
  -v $(pwd):/app \
  -v $(pwd)/logs:/app/logs \
  -p 8080:8080 \
  myapp python -m uvicorn main:app --reload --host 0.0.0.0

# Edit main.py on your host → uvicorn detects the change → reloads
# No docker build needed during development
```

3.4 tmpfs Mounts — RAM Storage, Gone on Restart

A tmpfs mount stores data in the host's RAM. It's fast, but disappears when the container stops. Use it for **sensitive temporary data that must never touch disk** — **tokens, session data, intermediate computation results**.

tmpfs for sensitive temporary data

```
# Store sensitive computation in RAM only
docker run -d \
  --name secure-processor \
  --tmpfs /tmp:size=512m,mode=1777 \
  --tmpfs /run/secrets:size=10m \
  myml-app

# --tmpfs /tmp:size=512m → /tmp is in RAM, max 512 MB
# mode=1777 → anyone can write (same as /tmp permission)

# In Docker Compose:
```

3.5 Volume Cheat Sheet — When to Use What

Situation	Use	Why
Database data (postgres, mysql, redis)	Named volume	Docker manages it, portable, survives container delete
Model weights during training output	Named volume	Write once, reference by name, easy to backup
Source code during development	Bind mount	Real-time sync with your editor
Config files (nginx.conf, etc.)	Bind mount	Easy to edit on host and test immediately
Sensitive tokens/session data	tmpfs	Never touches disk, gone on restart
ML training datasets (read-only)	Bind mount	Map your data directory directly in

3.6 Volume Commands You'll Use Every Day

Essential volume management commands

```
# List all volumes
docker volume ls

# Inspect a volume — find where it lives on disk
docker volume inspect pgdata
# Shows: Mountpoint: /var/lib/docker/volumes/pgdata/_data

# Remove a specific volume (must not be in use)
docker volume rm pgdata

# Remove all unused volumes (careful!)
docker volume prune

# Backup a volume to a tar file
docker run --rm \
  -v pgdata:/source:ro \
  -v $(pwd):/backup \
  ubuntu tar czf /backup/pgdata-backup.tar.gz -C /source .

# Restore from backup
docker run --rm \
  -v pgdata:/target \
  -v $(pwd):/backup \
  ubuntu tar xzf /backup/pgdata-backup.tar.gz -C /target
```

Project 2 — Feel Every Volume Type

Goal: Feel the difference between no volume, named volume, and bind mount.

Exercise 1 — Feel data loss: Run postgres, create a table, rm the container, recreate it, confirm the table is gone.

Exercise 2 — Named volume persistence: Repeat but with `-v pgdata:/var/lib/postgresql/data`. Delete container, recreate — data survives.

Exercise 3 — Dev bind mount: Create a Python script that prints 'Hello v1'. Run it with a bind mount. Edit the file to print 'Hello v2'. Run again — no rebuild needed, output changed instantly.

Goal: After this, volumes will be permanently wired into your instincts — not just memorized.

Docker Networking

How containers find and talk to each other

The Apartment Building Analogy

Your computer is an apartment building. Each container is one apartment with its own locked door. By default, apartments cannot communicate — they don't even know each other exists. A Docker **network** is the shared hallway. Once two containers are on the same hallway (network), they can knock on each other's door using the container's name as the door number. The **-p flag** is the building's front entrance — it lets visitors from outside reach a specific apartment.

4.1 The Default Bridge — Why It's Not Enough

When you install Docker, it creates a **network called bridge automatically**. All containers without a specified network go here. The problem: **containers on the default bridge cannot find each other by name.**

The default bridge has no DNS

```
# Start two containers on the default network
docker run -d --name web nginx
docker run -d --name db postgres -e POSTGRES_PASSWORD=pass

# Try to reach 'db' from 'web' by name
docker exec web ping db
# ping: db: Name or service not known
# FAILS — no DNS on the default bridge

# You'd have to use the IP directly (fragile, changes on restart)
docker inspect db --format '{{.NetworkSettings.IPAddress}}'
# e.g. 172.17.0.3
docker exec web ping 172.17.0.3
# Works, but hardcoding IPs is wrong
```

4.2 Custom Bridge Networks — The Right Way

A custom bridge network has **Docker's embedded DNS server** built in (at 127.0.0.11 inside containers). Every container on the network is automatically registered by its container name. This is how real applications work.

Custom bridge — DNS works by container name

```
# Step 1: Create a custom network
docker network create myapp-net
```

```
# Step 2: Start containers ON that network
```

```
docker run -d --name db --network myapp-net \
```

4.3 Port Publishing — Opening the Front Door

Containers are private by default. Nothing outside Docker can reach them. Port publishing creates a mapping from your host machine's port to a container's port.

Command	What it does
<code>docker run -p 8080:80</code>	Host port 8080 → container port 80. Accessible from anywhere.
<code>docker run -p 127.0.0.1:8080:80</code>	Host localhost:8080 → container port 80. NOT accessible from external machines. Safer.
<code>docker run -p 80</code>	Docker picks a random host port. Use 'docker port' to find it.
<code>docker run -p 5353:5353/udp</code>	UDP port mapping. Add /udp suffix for non-TCP protocols.

Port publishing examples

```
# See all port mappings for a container
docker port mycontainer
# 80/tcp -> 0.0.0.0:8080

# Verify with curl
curl http://localhost:8080

# Multiple ports (web + API + metrics)
docker run -d \
  -p 80:80 \
  -p 443:443 \
  -p 9090:9090 \
  --name myapp myimage
```

4.4 Network Isolation — Security by Design

Two custom networks are completely isolated from each other, even on the same host. This lets you design secure multi-tier architectures where your frontend cannot directly talk to your database.

Network isolation — frontend cannot reach database

```
# Create two isolated networks
docker network create frontend-net
docker network create backend-net

# nginx: only on frontend (public-facing)
docker run -d --name nginx --network frontend-net -p 80:80 nginx
```

4.5 Useful Network Commands

Network commands reference

```
# List networks
docker network ls

# Inspect a network – see which containers are connected
docker network inspect myapp-net

# Connect a running container to a network
docker network connect myapp-net mycontainer

# Disconnect
docker network disconnect myapp-net mycontainer

# Remove unused networks
docker network prune

# Debug DNS from inside a container
docker exec mycontainer cat /etc/resolv.conf
# nameserver 127.0.0.11 ← Docker's embedded DNS

docker exec mycontainer nslookup db
# Resolves 'db' to its container IP
```

Project 3 — 3-Tier Network Architecture

Goal: Build a 3-container application with proper network isolation.

Architecture:

- **nginx** on frontend-net, port 80 published to host
- **python-api** on both frontend-net and backend-net
- **postgres** on backend-net only

Verify:

- curl localhost → nginx responds
- nginx can reach python-api by name
- python-api can reach postgres by name
- nginx CANNOT reach postgres (docker exec nginx ping postgres fails)

This is the exact pattern used in production web applications.

Docker Compose

Running real multi-container applications in one file

The Screenplay Analogy

Running a multi-container app with docker run commands is like directing a movie by calling each actor individually every morning, telling them where to stand, what costume to wear, and who to talk to. Docker Compose is the **screenplay** — one document that describes everything. One command starts the whole production.

5.1 The compose.yml Structure

A compose.yml has three top-level sections. Every real app uses all three:

Complete compose.yml with all features

```
# compose.yml — complete structure
services:
  # Each service is a container
  web:
    image: nginx:1.25
    ports:
      - '80:80'
    networks:
      - frontend
    depends_on:
      api:
        condition: service_healthy

  api:
    build: .
    environment:
      - DB_HOST=db
      - DB_PORT=5432
    env_file:
      - .env
    networks:
      - frontend
      - backend
    healthcheck:
      test: ['CMD', 'curl', '-f', 'http://localhost:8080/health']
      interval: 10s
      timeout: 5s
      retries: 3
```

5.2 depends_on — Start Order and Health Conditions

A common beginner mistake: setting `depends_on` without a condition means Docker starts the container, not waits for the *service* to be ready. Postgres takes several seconds to initialise. Your API crashing because it connected before postgres was ready is a classic failure.

depends_on with health condition

```
# WRONG — just depends_on with no condition
depends_on:
  - db
# Starts db container, then immediately starts api
# Postgres might not be ready yet → api crashes with connection error

# RIGHT — wait for health check to pass
depends_on:
  db:
    condition: service_healthy
# Docker waits until db's healthcheck reports 'healthy'
# THEN starts the api service

# Conditions available:
# service_started — container started (default, not great)
# service_healthy — healthcheck passed (correct for databases)
# service_completed_successfully — for one-shot setup tasks
```

5.3 Environment Variables and .env Files

.env file pattern

```
# .env file (never commit this to git!)
DB_PASSWORD=supersecret123
API_KEY=sk-prod-abc123
LOG_LEVEL=INFO

# compose.yml — reference .env variables with ${VAR}
services:
  api:
    environment:
      DB_PASSWORD: ${DB_PASSWORD}
      API_KEY: ${API_KEY}
      LOG_LEVEL: ${LOG_LEVEL:-INFO}
      # ${VAR:-default} uses default if VAR not set

# .env is automatically loaded by docker compose
# You can also specify: docker compose --env-file prod.env up

# For multiple environments:
# .env          ← local development (in .gitignore)
```

5.4 Docker Compose Commands

Command	What it does
<code>docker compose up</code>	Start all services. Add <code>-d</code> for detached (background) mode.
<code>docker compose up --build</code>	Rebuild images before starting. Use after Dockerfile changes.
<code>docker compose down</code>	Stop and remove containers and networks. Volumes survive.
<code>docker compose down -v</code>	Stop and remove everything INCLUDING volumes. Wipes data.
<code>docker compose ps</code>	Show status of all services.
<code>docker compose logs -f</code>	Stream logs from all services. Add service name for one: <code>logs -f api</code>
<code>docker compose exec api bash</code>	Open a shell inside a running service.
<code>docker compose restart api</code>	Restart one service without touching others.
<code>docker compose pull</code>	Pull latest images for all services.
<code>docker compose config</code>	Validate and print the resolved <code>compose.yml</code> (with env substitution).

5.5 Project: Full ML Stack with Compose

Project 4 — Production ML Stack with Compose

Goal: A complete ML serving stack in one `compose.yml`

Services:

- **nginx** — reverse proxy, handles HTTPS termination, port 80
- **ml-api** — FastAPI app serving predictions, built from your Dockerfile
- **redis** — caching layer for prediction results
- **postgres** — stores prediction logs and metadata
- **prometheus** — scrapes metrics from ml-api

Requirements:

- ml-api waits for postgres to be healthy before starting
- All secrets in `.env` file
- postgres data in a named volume
- ml-api and postgres on `backend-net`
- nginx and ml-api on `frontend-net`

Success criteria: `docker compose up -d` starts everything. `curl localhost` returns a prediction. `docker compose down` brings everything down cleanly.

ML Stack `compose.yml`

```
# ML Stack compose.yml
services:
  nginx:
    image: nginx:1.25-alpine
    ports: ['80:80']
```


Production Docker

What professional teams actually do

6.1 Health Checks — Prove Your App Actually Works

A container can be 'running' but completely broken — process is alive but not serving requests. Health checks let Docker test your app and restart it automatically if it fails.

Health check in Dockerfile

```
# In Dockerfile:
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD curl -f http://localhost:8080/health || exit 1

# --interval=30s   check every 30 seconds
# --timeout=10s    if check takes over 10s, it fails
# --retries=3      fail 3 times in a row to be 'unhealthy'
# --start-period=10s  grace period on startup (add this for slow apps)

# Check health status
docker ps
# STATUS column shows: Up 2 minutes (healthy)
# or:                  Up 2 minutes (unhealthy)

docker inspect mycontainer --format '{{.State.Health.Status}}'
# healthy / unhealthy / starting

# See the last 5 health check results
docker inspect mycontainer --format '{{json .State.Health.Log}}'
```

6.2 Resource Limits — Protect Your Host

Without limits, one misbehaving container can exhaust all host resources. In production, always set memory limits. CPU limits are optional but recommended.

Resource limits

```
# Set limits in docker run
docker run -d \
    --memory=512m \
    --memory-swap=512m \
    --cpus=0.5 \
    --name myapp myimage
```

6.3 Restart Policies — Self-Healing Containers

Policy	When container restarts	Use case
no (default)	Never restarts	One-shot scripts, testing
always	Always, even if manually stopped	Critical services, databases
on-failure	Only if exit code is non-zero	Background jobs, batch processors
unless-stopped	Always, unless you explicitly stopped it	Most production services (recommended)

Restart policy example

```
docker run -d --restart unless-stopped --name db postgres:16

# 'unless-stopped' means:
# - Restarts if container crashes
# - Restarts if host reboots
# - Does NOT restart if you do: docker stop db
# This is the right choice for 95% of production services
```

6.4 Docker Registry — Sharing Images

A registry is where images live so other machines can pull them. Docker Hub is the public default. For production, you typically use a private registry (AWS ECR, GCP Artifact Registry, GitHub Container Registry).

Registry workflow

```
# Tag your image for a registry
# Format: registry/username/imagename:tag
docker tag myapp:latest yourusername/myapp:v1.0.0
docker tag myapp:latest yourusername/myapp:latest

# Push to Docker Hub
docker login
docker push yourusername/myapp:v1.0.0
docker push yourusername/myapp:latest

# Push to AWS ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS \
    --password-stdin 123456789.dkr.ecr.us-east-1.amazonaws.com

docker tag myapp:v1.0.0 \
  123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:v1.0.0
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:v1.0.0
```

6.5 Image Security Scanning

Security scanning

```
# Scan with Docker Scout (built into Docker Desktop)
docker scout cves myimage:latest

# Scan with Trivy (open source, widely used in CI)
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy image myimage:latest

# Trivy output categories:
# CRITICAL – fix immediately before production
# HIGH      – fix before next release
# MEDIUM    – fix in next sprint
# LOW       – track, fix when convenient

# Add to CI/CD pipeline – fail build on CRITICAL
trivy image --exit-code 1 --severity CRITICAL myimage:latest
```

Project 5 — Harden Your Production Stack

Goal: Make your ML Compose stack production-ready.

Add to your existing compose.yml:

- Health checks on all services
- restart: unless-stopped on all services
- Memory and CPU limits on ml-api
- Resource reservations for guaranteed minimums

Tag and push your ml-api image to Docker Hub.

Run a Trivy scan on your image and fix any CRITICAL vulnerabilities (usually means updating the base image or a specific package).

Success: docker compose ps shows all services as (healthy). docker scout cves shows no CRITICAL issues.

Debugging & Troubleshooting

How to diagnose and fix anything in Docker

Every Docker engineer spends significant time debugging. The difference between a junior and senior is how fast they can find what's wrong. This chapter gives you the systematic toolkit.

7.1 The Debugging Ladder — Start Here

When something breaks, run these commands in order. Most problems are solved by step 3.

The debugging ladder

```
# Step 1: Is the container running?
docker ps
docker ps -a
# -a shows stopped containers too
# Check STATUS column: Up? Exited? Restarting?

# Step 2: What did it print?
docker logs mycontainer
docker logs --tail 50 mycontainer
docker logs -f mycontainer
# -f follows in real time (like tail -f)

# Step 3: Is it healthy?
docker inspect mycontainer --format '{{.State.Status}}'
docker inspect mycontainer --format '{{.State.Health.Status}}'
docker inspect mycontainer --format '{{.State.ExitCode}}'

# Step 4: Get inside and look around
docker exec -it mycontainer bash
# or for alpine-based images (no bash):
docker exec -it mycontainer sh
```

7.2 Reading Logs Effectively

Reading logs

```
# Follow logs in real time
docker logs -f mycontainer

# Last 100 lines
docker logs --tail 100 mycontainer
```

7.3 Inspect — Know Everything About a Container

Inspect commands

```
# Full JSON dump of everything Docker knows
docker inspect mycontainer

# Extract specific fields with --format
docker inspect mycontainer --format '{{.State.Status}}'
docker inspect mycontainer --format '{{.NetworkSettings.Networks}}'
docker inspect mycontainer --format '{{.Mounts}}'
docker inspect mycontainer --format '{{.Config.Env}}'

# Inspect a network
docker network inspect myapp-net

# Inspect a volume
docker volume inspect pgdata

# Inspect an image — see layers
docker image inspect myimage:latest
docker history myimage:latest
```

7.4 Common Failures and Exact Fixes

Error Message	Cause	Fix
Port is already allocated	Something on your host uses port 8080	Change -p to a different host port, or stop the conflicting service
No such file or directory (COPY)	File not in build context, or path wrong	Check .dockerignore, verify file exists, check relative path
Permission denied	App tries to write to read-only location	Add WORKDIR, check USER, use a writable volume
OOMKilled	Container exceeded memory limit	Increase --memory limit or fix memory leak in app
connection refused (db)	App starts before database is ready	Add healthcheck + depends_on with condition: service_healthy
Name or service not known	Containers on different networks or wrong name	Put both on same custom network, use exact container name
image not found	Wrong image name, not pulled, or registry not logged in	docker pull first, check spelling, docker login for private registries
Exit code 1	App crashed on startup — read the logs	docker logs mycontainer — the error is always there

7.5 Advanced Debugging Techniques

Advanced debugging

```
# Debug a container that won't start
# Override CMD with bash to get inside before the app crashes
docker run -it --entrypoint bash myimage
# Now manually run your app and see the real error

# Debug networking from inside a container
# (install netshoot – a container full of network tools)
docker run -it --network myapp-net nicolaka/netshoot
# Inside: ping, curl, dig, nslookup, tcpdump, netstat all available

# Check what a container can reach
docker exec mycontainer curl -v http://db:5432
docker exec mycontainer nslookup db

# Watch container resource usage live
docker stats
docker stats mycontainer
# Shows CPU%, MEM USAGE/LIMIT, NET I/O, BLOCK I/O live

# Clean up everything (CAUTION – removes all stopped containers,
# unused networks, dangling images)
docker system prune
docker system prune -a
# -a also removes images not referenced by any container

# Check disk usage
docker system df
# Shows: Images, Containers, Volumes, Build cache sizes
```

Project 6 — Deliberately Break Things and Fix Them

Goal: Practice diagnosing 5 common failures deliberately.

Exercise 1 — Port conflict: Start nginx on port 80 twice. Fix it.

Exercise 2 — Missing env var: Start postgres without POSTGRES_PASSWORD. Read the log. Add the var.

Exercise 3 — Wrong network: Start two containers on different networks. Try to ping. Fix by connecting both to the same network.

Exercise 4 — OOMKill: Run a container with --memory=10m. Start a Python process that allocates 100 MB. Watch it get OOMKilled. Read docker inspect to confirm.

Exercise 5 — Crash on startup: Build an image with a typo in CMD (e.g., pythun main.py). Run it. Use docker logs to find and fix the error.

After this exercise, you will diagnose Docker problems faster than most people who've used it for years.

What Comes Next

You now have everything you need to use Docker confidently at a professional level. Here is the learning sequence that follows naturally from this book:

Next topic	Why it follows from Docker	Recommended starting point
Docker Swarm	Orchestrate Docker across multiple servers — the simpler alternative to Kubernetes	<code>docker swarm init</code> + a 3-node Swarm cluster
Kubernetes (K8s)	The industry standard for container orchestration at scale — what all MLOps platforms are built on	<code>kubecttl</code> + <code>kind</code> (local K8s) + your first Deployment
CI/CD Pipelines	Automatically build, test, scan, and push your Docker images on every git push	GitHub Actions with <code>docker/build-push-action</code>
Container Security	Deeper: non-root, read-only filesystems, seccomp profiles, AppArmor	<code>docker run --read-only</code> + user namespace remapping
Observability	Prometheus + Grafana to monitor your containers in production	<code>prometheus</code> + <code>grafana</code> in your compose stack

The one rule that separates experts from readers:

The Expert's Secret

Every concept in this book only becomes real when you **type the commands yourself**, hit an error, figure out why, and fix it. Reading builds awareness. Doing builds instinct. Run every code example. Do every project. Break things deliberately. The engineers who are truly dangerous with Docker are the ones who have seen the most failure modes — not read the most documentation.